KL UNIVERSITY

Hotel Booking and Occupancy Manager

A Java-based console application

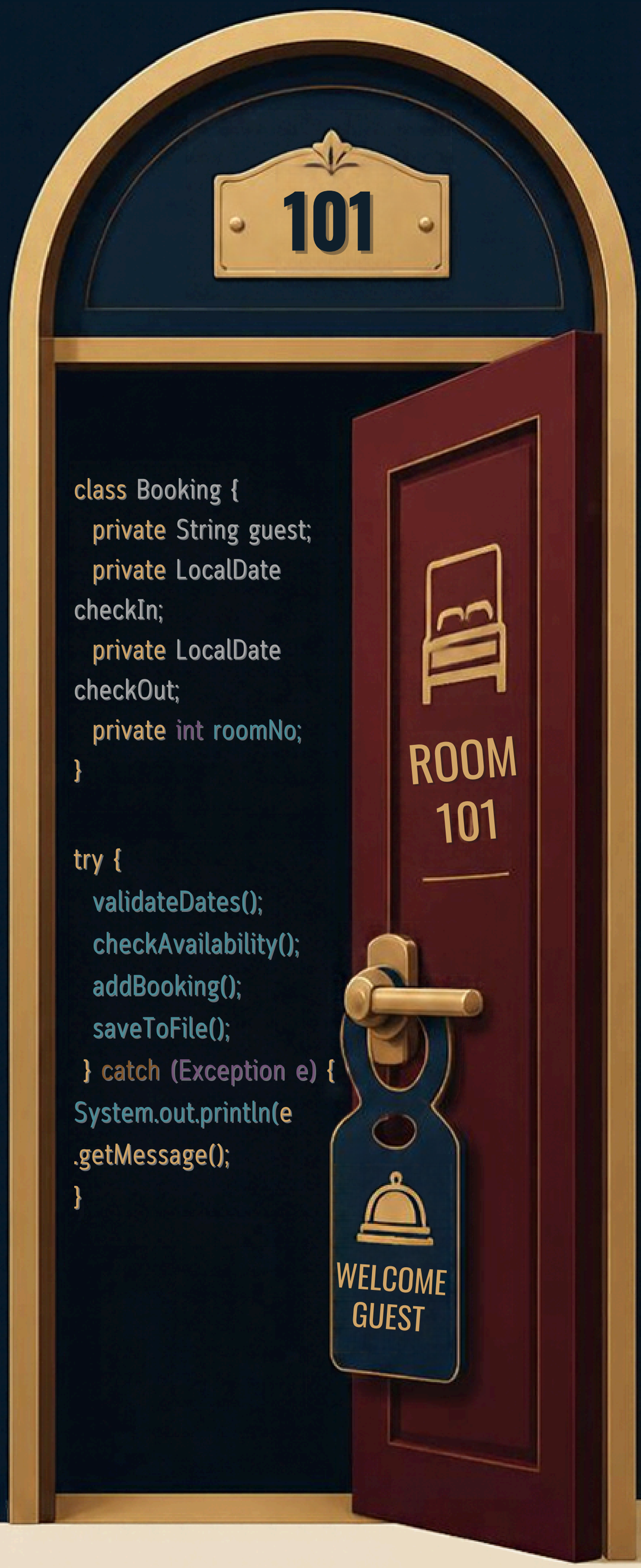
STUDENT	ROLL NUMBER
M. Amrutha Joanna	2620030246
[Alyaan Khan	2620030250



Hotel Booking and Occupancy Manager

A Java-Based Console Application

- **Prevents double-bookings**
- **Validates date ranges**
- **Manages rooms, guests, and reservations**
- **Forecasts 7-day occupancy**
- **Persists bookings with file I/O**



Core Java Concepts

OOP Exception Handling Collections Comparators NIO File I/O

Academic Project | 15-10-2026

KL UNIVERSITY

Abstract

A lightweight console system that turns hotel inventory into validated, persistent reservations.

- Automates reservations and prevents double-bookings.
- Maintains live room type, price and availability.
- Registers guests and validates booking date ranges.
- Produces a 7-day occupancy forecast by room type.
- Saves booking records to a text file for retention.

JAVA FUNDAMENTALS

PRIMITIVES / CONTROL FLOW
METHODS / OOP
EXCEPTIONS / STRINGS
FILE I/O / COLLECTIONS
COMPARATOR SORTING

Understanding Problem Statement

THE CURRENT REALITY

Manual registers create double-bookings, slow check-ins and an unclear availability view. Staff cross-check pages—inviting errors and lost revenue.

THE SYSTEM MUST

- Store room type, AC status and price
- Register guest contact details
- Validate date ranges and prevent overlaps
- Reject past or inverted dates
- Report occupancy percentage
- Persist bookings between sessions

GAP → Paper and spreadsheets offer no enforced overlap logic or forward occupancy forecast.

CO 1 Topics: From Problem to Program

DECOMPOSE

Rooms

Guests

Bookings

Reports

ALGORITHM DESIGN

```
input  →  validate
check overlap  →  confirm
```

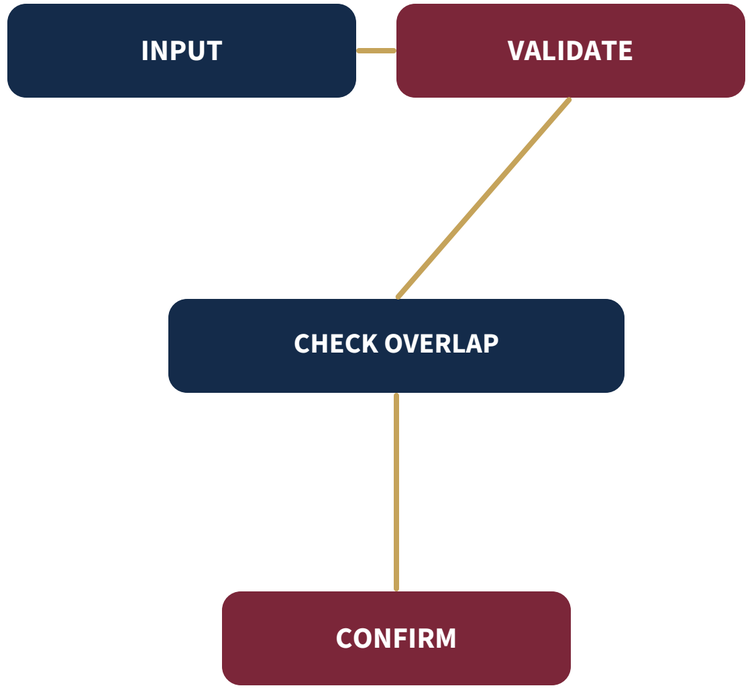
JAVA FOUNDATIONS

```
int  roomNo, id
double  price
boolean  ac
static final String FILE
= "bookings.txt"
```

Operators: arithmetic, relational, logical, ternary • Typecasting for occupancy • Scanner input • printf output

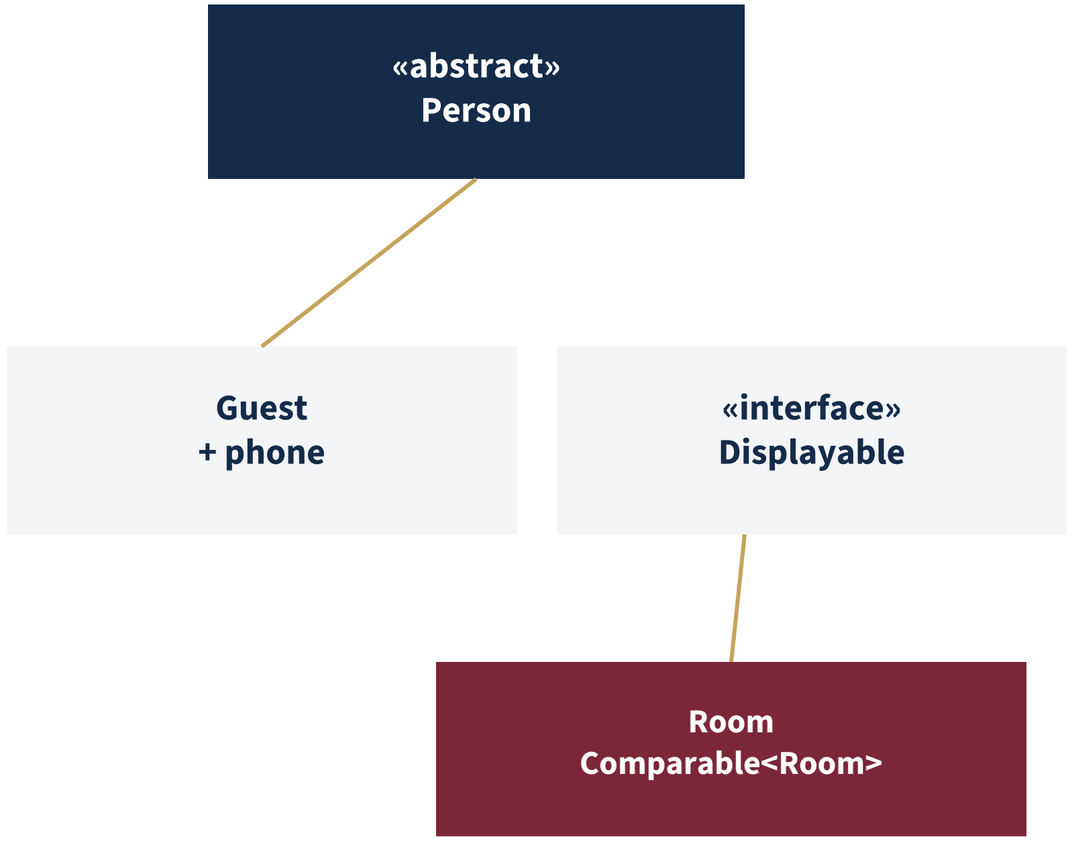
CO 2 Topics: Control Flow & Modular Programming

- if / if-else / switch for menu and validation
- while(true) main menu; for and enhanced for loops
- findRoom(), isBooked(), validateDates()
- Overloaded read(), readInt(), readDate()
- Collection traversal for search and overlap checks
- forecast() nests loops by day and room type



CO 3 Topics: OOP Modelling

- Classes: Room, Guest, Booking
- Parameterized constructors and this keyword
- Encapsulation with private fields and getters
- Person is an abstract base; Guest extends Person
- Displayable defines the display() contract
- Room implements Displayable and Comparable<Room>
- toString() is overridden
- Booking.counter auto-generates IDs



CO 4 Topics: Exception Handling

Checked and unchecked exceptions are handled deliberately. Custom checked exceptions encode business rules; throw flags conflicts, while validateDates() declares throws. try-catch and multi-catch keep the menu alive with friendly messages. File loading tolerates missing or malformed records.

RULES FAIL SAFELY — THE APPLICATION DOES NOT.

RoomUnavailableException

InvalidDateException

```
try {
  validateDates(in, out);
  book(room, guest);
} catch (InvalidDateException |
         RoomUnavailableException e) {
  showFriendlyMessage(e);
}
```

CO 5 Topics: Strings, Library Algorithms & File I/O

STRINGS
toLowerCase() · contains()
split() · trim()
StringBuilder
equals() vs ==

SORTING
Comparable: room number
Comparator.comparingDouble()
Room::getPrice
Collections.sort()

FILE I/O
Files.write()
newBufferedReader()
Files.exists()
try-with-resources

Booking objects → Booking::toFileLine → bookings.txt → split(";") on load

CO 6 Topics: Collections & Functional Style



ArrayList · HashMap · HashSet · generics · autoboxing · entrySet() · method references

CO 6 Continued / Design Overview

CLASS	RESPONSIBILITY
`Person (abstract)`	Base for people
`Guest extends Person`	Phone + Displayable
`Room`	Details + Comparable
`Booking`	Links guest, room, dates
`HotelApp`	Menu, state, helper methods

KEY ALGORITHMS

```
OVERLAP
room.checkIn < out
&& in < booking.checkOut

OCCUPANCY
(distinct booked rooms ÷ total rooms) × 100

PERSISTENCE
one semicolon-separated line per booking
split(";") when loading
```

Conclusion

A practical Java system built around one clear promise:

every valid guest gets the right room—once

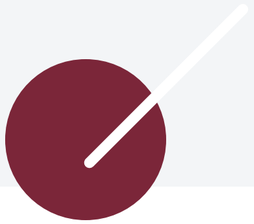
Control flow powers the menu. OOP structures the domain. Exceptions enforce business rules. Strings and file I/O preserve bookings. Collections and Comparators keep data flexible. Together, they create a lightweight, extensible manager that prevents double-bookings and tracks occupancy.

NEXT

Check-in / out

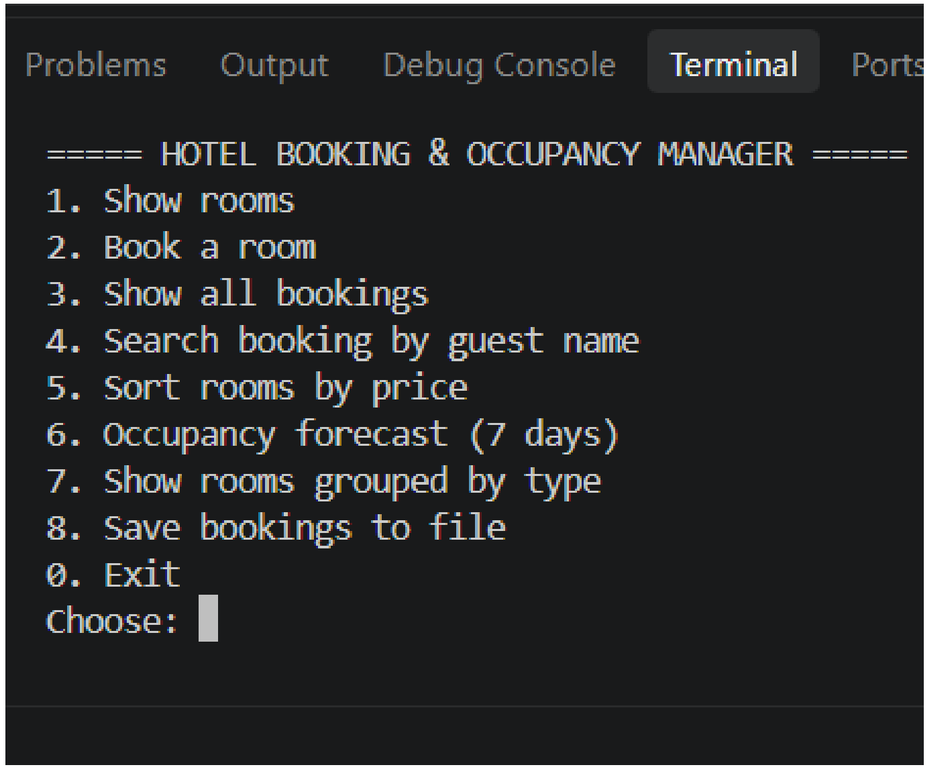
Billing

Graphical interface



JAVA FOUNDATIONS → REAL-WORLD RELIABILITY

Screenshots of the Output



Geo Tag Photo and Github link

https://github.com/mamruthajoanna-spec/PSPJ-project-expo